



МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)»

Институт №3 «Системы управления, информатика и электроэнергетика»

Кафедра 304 «Вычислительные машины, системы и сети»

Курсовая работа
по дисциплине «Проектная деятельность»

Выполнил
студент группы МЗО-209БВ-24

Плыкин А.В.

Принял
Максимов Е.Д.

Москва
2026

Оглавление

1. Проблема и выбранный метод решения	3
2. Стек технологий	3
3. Архитектура сервиса/программы	3
4. Детали реализации.....	3
5. Результаты.....	4
6. Вывод	5
Приложение.....	6

1. Проблема и выбранный метод решения

Проблема:

Отсутствие у команды собственной программной платформы для экспериментов с 3D-графикой. Использование готовых коммерческих движков вроде Unity или Unreal Engine избыточно для учебных и исследовательских задач. Они скрывают внутренние механизмы рендеринга, ограничивают низкоуровневый контроль и не позволяют глубоко изучить принципы работы графического конвейера. Требовался лёгкий, полностью контролируемый и расширяемый инструмент, который можно адаптировать под любую задачу — от визуализации моделей до тестирования новых шейдеров.

Метод решения:

Разработать собственный 3D-движок на C++ с использованием OpenGL 3.3 в качестве графического API. Архитектура строится на компонентно-ориентированном подходе: все объекты сцены унифицированы общим интерфейсом IModel, ресурсы инкапсулированы в классы с автоматическим управлением памятью, UI-редактор реализован через Dear ImGui. Такой подход обеспечивает модульность, возможность независимой разработки компонентов и лёгкое расширение функциональности без модификации ядра.

2. Стек технологий

- Язык программирования: C++17
- Графический API: OpenGL 3.3 (Core Profile, GLSL-шейдеры, VAO/VBO/EBO)
- Загрузка OpenGL: GLAD
- Математическая библиотека: GLM (трансформации, векторы, матрицы, проекции)
- Оконный менеджмент и ввод: SDL3
- Пользовательский интерфейс: Dear ImGui (бэкенды ImGui_ImplSDL3, ImGui_ImplOpenGL3)
- Загрузка изображений: SDL3_image
- Рендеринг шрифтов: FreeType
- Сериализация: Ручной парсинг формата Wavefront OBJ

3. Архитектура сервиса/программы

(Общее описание архитектуры — см. отчёт О. Юфатова, раздел 3)

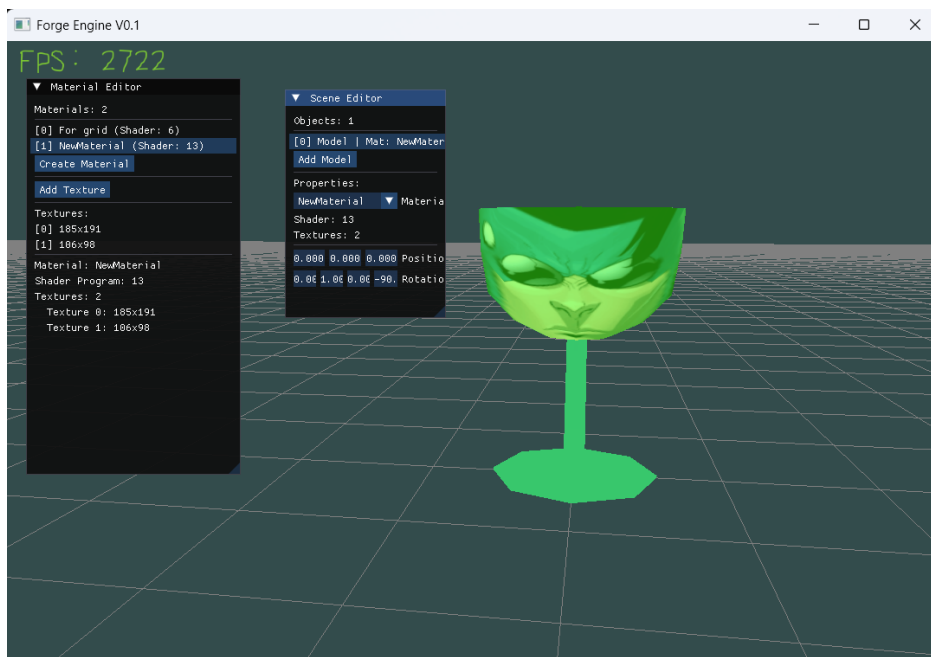
4. Детали реализации

Мой вклад был полностью сфокусирован на двух направлениях: UI-редактор и система рендеринга текста.

- **Разработка пользовательского интерфейса (RenderUI, RenderSceneEditor, ...):**
 - Осуществлена инициализация ImGui в методе Engine::Initialize с настройкой стилей под DPI-масштаб дисплея.
 - Разработан многооконный редактор в методе RenderUI, состоящий из окон "Scene Editor" и "Material Editor".

- Реализован функционал управления сценой: список объектов с отображением их типа, назначенного материала и позиции. Контекстные меню позволяют удалять модели через `RenderRemoveModelPopup`.
- Создан инспектор объектов: если в списке выбран объект, внизу окна появляется Combo-виджет для выбора материала из общего списка и элементы редактирования свойств самого объекта (через вызов `model->RenderProperties()`).
- В "Material Editor" реализовано создание новых материалов с указанием путей к шейдерам (`RenderCreateMaterialPopup`) и управление текстурами (`RenderAddTexturePopup`, `RenderRemoveTexturePopup`).
- **Разработка системы рендеринга текста (`TextRenderer`):**
 - Разработан класс `TextRenderer`, инкапсулирующий работу с FreeType.
 - В конструкторе реализован механизм загрузки шрифта и предварительной растеризации всех 128 ASCII-символов в отдельные текстуры OpenGL. Для каждого символа (`Character`) сохранены метрики: размер, `bearing` и `advance`.
 - Ключевая особенность — использование одного динамического VBO. В `RenderText` для каждого символа рассчитывается его положение и размер с учётом масштаба, после чего данные вершин обновляются в VBO через `glBufferSubData`. Это исключает создание нового буфера на каждый символ, что является эффективным методом.
 - Рендеринг текста выведен в отдельный проход с отключённым тестом глубины и использованием ортографической проекции, чтобы FPS-счётчик всегда отображался поверх 3D-сцены.

5. Результаты



6. Вывод

Проект решил поставленную проблему: создан полностью функциональный 3D-движок-песочница с редактором сцены. Заложенная архитектура позволяет легко добавлять новые типы объектов без переписывания ядра — достаточно создать класс-наследник IModel. Полученный инструмент пригоден как для учебных экспериментов с графическим конвейером, так и для прототипирования более сложных приложений.

Приложение

Engine.h (ядро движка)

```
class Engine {
public:
    bool Initialize(int width, int height, const char* title);
    void Run();
    void Shutdown();
private:
    SDL_Window* m_window = nullptr;
    SDL_GLContext m_glContext = nullptr;
    std::vector<std::unique_ptr<IModel>> m_objects;    // все объекты сцены
    std::vector<std::unique_ptr<Material>> m_materials; // все материалы
    std::unique_ptr<Grid> m_grid;
    std::unique_ptr<TextRenderer> m_textRenderer;

    // Камера
    glm::vec3 m_cameraPos{0.0f, 2.0f, 3.0f};
    glm::vec3 m_cameraFront{0.0f, 0.0f, -1.0f};
    float m_yaw = -90.0f, m_pitch = 0.0f;
    bool m_cursorCaptured = false;

    void ProcessEvents();
    void Update(float deltaTime);
    void Render();
    void RenderUI();
    void RenderSceneEditor();
    void RenderMaterialEditor();
    IModel* AddModel(const std::string& path, const glm::vec3& pos, Material* mat);
    Material* CreateMaterial(const std::string& name, const std::string& vert, const std::string& frag);
};
```

IModel.h (интерфейс объектов сцены)

```
class IModel {
public:
    virtual ~IModel() = default;
    virtual void Render(glm::mat4& view, glm::mat4& proj) = 0;
    virtual std::string& GetTypeName() = 0;
    virtual glm::vec3 GetPosition() = 0;
    virtual Material* GetMaterial() = 0;
    virtual void SetMaterial(Material* mat) = 0;
    virtual void RenderProperties() {}
};
```

Material.h (управление шейдерами и текстурами)

```
class Material {
    GLuint shaderProgram = 0;
    std::string name;
public:
    std::vector<std::shared_ptr<Texture>> textures;
    Material(const std::string& name);
    GLuint CompileShaderProgram(const char* vertSrc, const char* fragSrc);
    bool loadShader(const std::string& vertPath, const std::string& fragPath);
    void bindTextures(); // автоматически биндит все текстуры на юниты
    void use();
    GLuint getShaderProgram() const { return shaderProgram; }
};
```

Model.h (3D-модель из OBJ)

```
class Model : public IModel {
    GLuint VAO, VBO, EBO;
    Material* material;
    std::vector<float> vertices;
    std::vector<unsigned int> indices;
    glm::vec3 position{0,0,0};
    glm::vec4 rotation{0, 1, 0, -90};
public:
    bool LoadObjModel(std::string& path);
    void Render(glm::mat4& view, glm::mat4& proj) override;
    void RenderProperties() override; // ImGui: DragFloat3, DragFloat4
};
```

ObjParser.h (парсер Wavefront OBJ)

```
class ObjParser {
    std::vector<glm::fvec3> temp_positions;
    std::vector<glm::fvec2> temp_texcoords;
    std::vector<glm::fvec3> temp_normals;
    std::vector<float> vertices; // [x,y,z, u,v, nx,ny,nz] * 8
    std::vector<unsigned int> indices;

    struct VertexKey { int pos, tex, norm; };
    std::map<VertexKey, unsigned int> vertexCache; // переиспользование вершин

    unsigned int GetOrCreateVertex(int posIdx, int texIdx, int normIdx);
    void ParseFace(const std::string& line);
public:
    bool LoadObjModel(const std::string& filename);
    const std::vector<float>& GetVertices() const;
    const std::vector<unsigned int>& GetIndices() const;
};
```

Engine.cpp — ключевые фрагменты

Главный цикл:

```
void Engine::Run() {
    m_running = true;
    float lastTime = SDL_GetTicks() / 1000.0f;
    while (m_running) {
        float deltaTime = SDL_GetTicks() / 1000.0f - lastTime;
        lastTime += deltaTime;
        ProcessEvents();
        Update(deltaTime);
        Render();
        RenderUI();
        SDL_GL_SwapWindow(m_window);
    }
}
```

Захват мыши для FPS-камеры:

```
if (e.type == SDL_EVENT_MOUSE_BUTTON_DOWN &&
    e.button.button == SDL_BUTTON_RIGHT && keys[SDL_SCANCODE_LCTRL]) {
    m_cursorCaptured = true;
    SDL_SetWindowRelativeMouseMode(m_window, true);
}
if (e.type == SDL_EVENT_MOUSE_MOTION && m_cursorCaptured) {
    m_yaw += e.motion.xrel * 0.1f;
    m_pitch -= e.motion.yrel * 0.1f;
    // пересчёт m_cameraFront из углов Эйлера
}
```

Рендеринг сцены:

```
void Engine::Render() {
    glm::mat4 view = glm::lookAt(m_cameraPos, m_cameraPos + m_cameraFront, m_cameraUp);
    glm::mat4 proj = glm::perspective(45°, aspect, 0.1f, 100.0f);
    m_grid->Render(view, proj);
    for (auto& obj : m_objects) obj->Render(view, proj);
    // отрисовка FPS через ортографическую проекцию
}
```

Model.cpp — рендеринг модели

```
void Model::Render(glm::mat4& view, glm::mat4& proj) {
    glm::mat4 model = glm::translate(glm::mat4(1.0f), position);
    model = glm::rotate(model, radians(rotation.w), vec3(rotation));
    // передача матриц в шейдер
    glUniformMatrix4fv(/* uniModel, uniView, uniProjection */);
    material->bindTextures();
    glBindVertexArray(VAO);
    glDrawElements(GL_TRIANGLES, indices.size(), GL_UNSIGNED_INT, 0);
}
```

ObjParser.cpp — GetOrCreateVertex

```
unsigned int ObjParser::GetOrCreateVertex(int posIdx, int texIdx, int normIdx) {
    VertexKey key{posIdx, texIdx, normIdx};
    auto it = vertexCache.find(key);
    if (it != vertexCache.end()) return it->second; // переиспользование

    unsigned int idx = vertices.size() / 8;
    vertexCache[key] = idx;
    // упаковка: pos.xyz, tex.uv, norm.xyz
    const auto& p = temp_positions[posIdx];
    const auto& t = temp_texcoords[texIdx];
    const auto& n = temp_normals[normIdx];
    vertices.insert(vertices.end(), {p.x,p.y,p.z, t.x,t.y, n.x,n.y,n.z});
    return idx;
}
```

TextRenderer.cpp — ключевой фрагмент

```
TextRenderer::TextRenderer(const std::string& pathFont) {
    // 1. Инициализация FreeType
    // 2. Загрузка шрифта, установка размера 48px
    // 3. Для каждого из 128 ASCII-символов:
    for (unsigned char c = 0; c < 128; c++) {
        FT_Load_Char(face, c, FT_LOAD_RENDER);
        // растеризация глифа в GL_RED текстуру
        // сохранение метрик: size, bearing, advance
        Characters[c] = {texture, size, bearing, advance};
    }
}

void TextRenderer::RenderText(const std::string& text) {
    for (char c : text) {
        Character ch = Characters[c];
        // расчёт позиции квада
        // обновление VBO через glBufferSubData
        glDrawArrays(GL_TRIANGLES, 0, 6);
        x += (ch.advance >> 6) * scale;
    }
}
```